



Skin Cancer Detection System

A Deep Learning approach to automated skin lesion classification using Convolutional Neural Networks — built on the HAM10000 dataset across 7 disease categories.

HEALTHCARE AI

SUPERVISED LEARNING

MULTICLASS CLASSIFICATION

Project Overview

This project implements a **Skin Cancer Detection System** that automatically classifies skin lesion images into one of seven disease categories. By leveraging Convolutional Neural Networks (CNNs) trained on the widely used **HAM10000 dataset**, the system brings Healthcare AI into a practical, student-implementable framework. The core problem is a **multiclass image classification** task — the model receives a raw skin lesion image as input and outputs the most probable disease class.

Key Project Facts

- **Domain:** Healthcare AI
- **Problem Type:** Multiclass Image Classification
- **Learning Type:** Supervised Learning
- **Dataset:** HAM10000
- **Total Classes:** 7 Disease Categories
- **Input:** Skin lesion images (128×128 RGB)
- **Output:** Predicted skin disease class

Why This Matters

Skin cancer is one of the most prevalent and, when caught early, most treatable forms of cancer. Manual diagnosis relies on dermatologist expertise, which is not always accessible. An AI-powered detection system can assist clinicians by providing a fast, preliminary classification of suspicious lesions — especially valuable in resource-limited settings.

This project demonstrates the practical application of supervised deep learning, using real-world medical image data, standard preprocessing pipelines, and a CNN architecture that mirrors industry practices.

The 7 Disease Classes

The HAM10000 ("Human Against Machine with 10,000 training images") dataset contains dermoscopic images labelled across seven clinically distinct skin disease categories. Understanding each class is essential to appreciating the complexity of the classification problem — some are benign, others are malignant.

akiec

Actinic Keratoses — Pre-cancerous lesions caused by sun damage. Can progress to squamous cell carcinoma if untreated.

bcc

Basal Cell Carcinoma — Most common form of skin cancer. Rarely metastasises but causes local tissue damage.

bkl

Benign Keratosis — Non-cancerous skin growths including seborrheic keratoses. Harmless but often confused with malignant lesions.

df

Dermatofibroma — Benign fibrous nodule. Commonly found on legs; typically harmless and does not require treatment.

mel

Melanoma — Most dangerous skin cancer with high metastatic potential. Early detection is critical for survival outcomes.

nv

Melanocytic Nevi — Common moles (benign). The most abundant class in HAM10000, leading to class imbalance challenges.

vasc

Vascular Lesions — Includes angiomas and haemangiomas. Benign vascular tissue abnormalities, visually distinct in dermoscopy.

⚠ The dataset is **heavily imbalanced** — Melanocytic Nevi (nv) accounts for the majority of samples, while classes like df and vasc have far fewer samples. This imbalance must be addressed during training.

Data Preprocessing Pipeline

Raw images from the HAM10000 dataset cannot be fed directly into a neural network. A structured preprocessing pipeline ensures data consistency, correct formatting, and compatibility with the CNN model's expected input shape of (128, 128, 3).

1 Image Reading
Each image is loaded using `cv2.imread()`. OpenCV reads images in BGR format by default — a critical detail that must be corrected before further processing.

2 Colour Conversion
`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` converts images from BGR to RGB colour space, ensuring compatibility with standard visualisation libraries and correct feature learning.

3 Image Resizing
All images are resized to **128×128 pixels** using `cv2.resize()`. Fixed input dimensions are mandatory for batch processing in a CNN. Larger sizes increase accuracy but also memory and compute costs.

4 Label Encoding
Class names (strings like "mel", "bcc") are converted to integer indices using `LabelEncoder()` from Scikit-learn. This is a prerequisite for one-hot encoding.

5 One-Hot Encoding
`to_categorical()` from Keras converts integer labels into binary vectors of length 7. For example, class index 4 becomes `[0,0,0,0,1,0,0]` — required for categorical crossentropy loss.

Dataset Split & Data Augmentation

Train-Test Split

The preprocessed dataset is divided into training and testing subsets using Scikit-learn's `train_test_split()` function with `test_size=0.2`.

This means:

- **80% Training Data** — used to teach the CNN model
- **20% Testing Data** — held out strictly for final evaluation

The split is performed after shuffling to prevent ordering bias. A `random_state` seed ensures reproducibility across runs.

- ① The training set is what the model "sees" during learning. The test set is unseen data used to evaluate how well the model generalises to new images.

Data Augmentation

Since medical image datasets are limited and imbalanced, **data augmentation** using Keras's `ImageDataGenerator()` artificially expands the training set by applying random transformations to existing images. This prevents overfitting and improves model generalisation.



Rotation

Images rotated by a random degree — skin lesions appear at all orientations clinically.



Flips

Both horizontal and vertical flips double effective dataset size with minimal distortion.



Zoom

Random zoom in/out simulates varying camera distances during image capture.

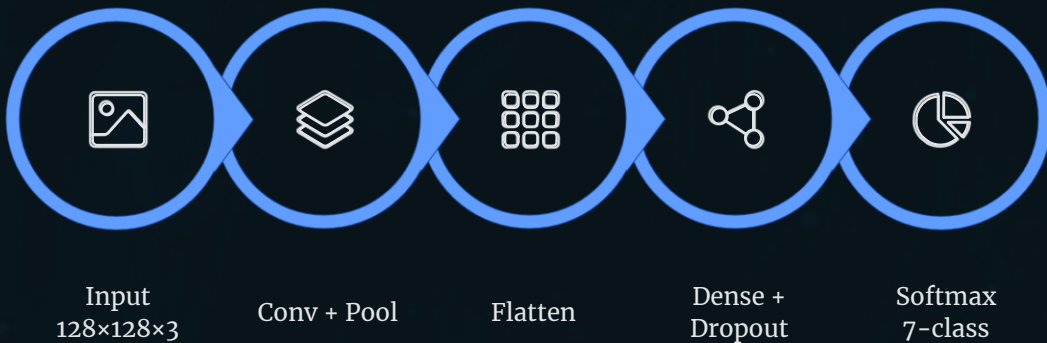


Brightness

Brightness range variation accounts for different lighting conditions during dermoscopy.

CNN Architecture & Layers

The core of this project is a Convolutional Neural Network (CNN) built using TensorFlow and Keras's `Sequential()` API. CNNs are specifically designed for image data — they learn spatial hierarchies of features, from simple edges in early layers to complex patterns like lesion borders and colour variations in deeper layers.



The architecture begins with three progressively deeper convolutional blocks, each followed by max pooling, which together act as a powerful feature extractor. The flattened output is then passed to fully connected Dense layers that perform the final disease classification. Dropout regularisation is applied to prevent the model from memorising training data.

Layer	Filters / Units	Activation	Purpose
Conv2D (Layer 1)	32 filters, 3×3	ReLU	Detect low-level features: edges, corners
MaxPooling2D	—	—	Reduce spatial dimensions, retain key features
Conv2D (Layer 2)	64 filters, 3×3	ReLU	Detect mid-level patterns: textures, shapes
MaxPooling2D	—	—	Further downsampling for efficiency
Conv2D (Layer 3)	128 filters, 3×3	ReLU	Detect high-level features: lesion morphology
MaxPooling2D	—	—	Final spatial reduction before classification
Flatten	—	—	Convert 2D feature maps → 1D vector
Dense	Fully connected	ReLU	High-level reasoning and classification
Dropout	0.5 rate	—	Randomly drops 50% neurons — prevents overfitting
Dense (Output)	7 neurons	Softmax	Output probability distribution over 7 classes

Activation Functions & Training Configuration

Activation Functions

Two activation functions are used at different stages of the network, each serving a specific mathematical purpose:

ReLU — Hidden Layers

Rectified Linear Unit: $f(x) = \max(0, x)$. Used in all convolutional and dense hidden layers. ReLU introduces non-linearity while remaining computationally simple. It solves the **vanishing gradient problem** that plagued earlier sigmoid activations, enabling faster and more stable training of deep networks.

Softmax — Output Layer

Softmax converts raw output scores (logits) into a **probability distribution** across all 7 classes, where all probabilities sum to 1.0. Example output: `mel=85%, bcc=10%, nv=5%`. The class with highest probability is taken as the prediction.

Training Configuration

- **Optimizer:** Adam — Adaptive Moment Estimation. Automatically adjusts learning rates per parameter, combining momentum and RMSProp. Most popular choice for deep learning tasks.
 - **Loss Function:** Categorical Crossentropy — the standard loss for multiclass classification with one-hot encoded labels. Penalises confident wrong predictions more heavily.
 - **Metric:** Accuracy — percentage of correctly classified images across all 7 classes.
 - **Batch Size:** 16 — 16 images are processed together in each gradient update step. Smaller batches give noisier but often better generalising updates.
 - **Epochs:** 10 — the entire training dataset is passed through the network 10 complete times. Each epoch updates weights based on accumulated gradients.
-
- ❑ Class imbalance is handled using `compute_class_weight()` from Scikit-learn, which assigns higher weights to underrepresented disease classes (e.g., `df`, `vasc`), so the model pays proportionally more attention to rare but clinically important conditions.

Model Evaluation & Visualisation

After training, the model is rigorously evaluated using multiple complementary methods to understand not just overall accuracy, but per-class performance — which is critical in a medical context where misclassifying melanoma as a benign lesion carries serious consequences.

Evaluation Methods

Accuracy Score

The percentage of images correctly classified across all 7 disease classes. Provides a high-level summary of model performance on the held-out 20% test set.

Confusion Matrix

A 7×7 matrix (via `confusion_matrix()`) showing, for each true class, how many samples were predicted as each possible class. Reveals which diseases are being confused with each other — invaluable for diagnosing model weaknesses.

Classification Report

Generated by `classification_report()`, this provides per-class **Precision** (of all predicted positives, how many were correct), **Recall** (of all actual positives, how many were found), and **F1-Score** (harmonic mean of precision and recall). Essential for evaluating performance on imbalanced classes.

Visualisation Graphs

Countplot

`sns.countplot()` — Visualises disease class distribution. Shows the imbalance between classes like nv (thousands of samples) vs df or vasc (very few samples).

Accuracy Graph

`plt.plot()` — Plots training vs validation accuracy across epochs. If validation accuracy lags significantly behind training accuracy, overfitting is indicated.

Loss Graph

`plt.plot()` — Plots training vs validation loss. Diverging loss curves (training loss drops, validation loss rises) are a classic overfitting signal.

Confusion Heatmap

`sns.heatmap()` — Renders the confusion matrix as a colour-coded grid. Darker diagonal cells indicate strong per-class prediction accuracy.

Libraries Used & Technology Stack

This project leverages a carefully chosen set of Python libraries, each serving a specific role in the data science and deep learning pipeline. Together, they form the standard toolkit used in industry-grade Healthcare AI projects.



NumPy

Provides the foundational **N-dimensional array (ndarray)** structure. All image pixel data is stored and manipulated as NumPy arrays. Essential for fast numerical operations like normalisation (dividing pixel values by 255.0 to scale to [0,1] range).



Pandas

Used for **loading and managing the HAM10000 metadata CSV**, which contains image IDs and their corresponding disease class labels. Pandas DataFrames make it easy to filter, merge, and prepare tabular data before image loading.



OpenCV (cv2)

The primary **image processing library**. Handles reading raw image files (`cv2.imread()`), colour space conversion from BGR to RGB (`cv2.cvtColor()`), and resizing all images to a uniform 128×128 pixel resolution (`cv2.resize()`).



Matplotlib & Seaborn

Used for all **data visualisation**. Matplotlib's `plt.plot()` renders accuracy and loss curves across epochs. Seaborn's `sns.countplot()` shows class distribution, and `sns.heatmap()` renders the confusion matrix in an interpretable colour-coded format.



Scikit-learn

Provides essential **ML preprocessing and evaluation utilities**: `LabelEncoder()` for converting string class names to integers, `train_test_split()` for dataset partitioning, `compute_class_weight()` for handling class imbalance, and `classification_report()` plus `confusion_matrix()` for evaluation.



TensorFlow & Keras

The **deep learning backbone** of the project. TensorFlow provides the computational graph and GPU acceleration. Keras (as TensorFlow's high-level API) offers the `Sequential()` model builder, all CNN layer classes (`Conv2D`, `MaxPooling2D`, `Dense`, `Dropout`), `ImageDataGenerator()` for augmentation, and `to_categorical()` for one-hot encoding.

Complete Project Workflow & Final Output

Bringing together all components — preprocessing, augmentation, model architecture, training, evaluation, and deployment — the project follows a clean end-to-end pipeline. The trained model is saved as `skin_cancer_model.h5` for future inference without retraining.

01

Load Dataset

Read HAM10000 metadata CSV using Pandas. Map image IDs to file paths and disease class labels.

02

Read & Preprocess Images

Load each image using `cv2.imread()`, convert BGR→RGB, resize to 128×128, and normalise pixel values to [0,1].

03

Encode Labels

Apply `LabelEncoder()` to convert class strings to integers, then `to_categorical()` for one-hot encoded vectors of length 7.

04

Split Dataset

Use `train_test_split(test_size=0.2)` to divide data — 80% for training, 20% reserved for final evaluation.

05

Apply Data Augmentation

Configure `ImageDataGenerator()` with rotation, zoom, horizontal/vertical flip, and brightness variation to expand effective training data.

06

Build CNN Model

Construct `Sequential()` CNN with 3 Conv2D+MaxPooling blocks (32→64→128 filters), Flatten, Dense, Dropout(0.5), and 7-neuron Softmax output.

01

Handle Class Imbalance

Compute class weights using `compute_class_weight()` and pass them to the model's `fit()` call so rare disease classes receive proportionally higher attention during training.

02

Train the Model

Train using Adam optimiser, Categorical Crossentropy loss, batch size 16, for 10 epochs. Monitor training and validation accuracy/loss at each epoch.

03

Evaluate Performance

Assess model on the held-out test set using accuracy score, confusion matrix, classification report (precision, recall, F1), and visualisation graphs.

04

Predict Disease Class

For a new skin lesion image, run `model.predict()` to get class probabilities, then `np.argmax()` to retrieve the highest-confidence predicted disease class.

05

Save Trained Model

Persist the fully trained CNN to disk as `skin_cancer_model.h5` using Keras's `model.save()`. This HDF5 file stores architecture, weights, and optimiser state — ready for deployment or future inference.



Project Outcome: A fully trained CNN capable of classifying skin lesion images into 7 disease categories, evaluated with industry-standard metrics, and saved as a reusable model file — demonstrating a complete, production-aware deep learning workflow.

SKIN CANCER

DETECTION SYSTEM

Deep Learning approach for automated lesion classification using

Transfer Learning on the HAM10000 dataset.

```
# ResNet50 Base Implementation
from tensorflow.keras.applications import ResNet50
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)
```

CNN ARCHITECTURE

The Transfer Learning Pipeline

Leveraging a pre-trained ResNet50 backbone enables the model to identify complex medical features with high precision.

```
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(512, activation='relu'),
    Dropout(0.4),
    Dense(num_classes, activation='softmax')
])
```

- **Input:** 224x224 RGB Images
- **Base:** Frozen weights for feature extraction
- **Head:** Custom Dense layers for 7 classes

TRAINING CONFIGURATION

Preprocessing & Augmentation

To combat class imbalance, random transformations are applied during training.

```
train_datagen = ImageDataGenerator(  
    preprocessing_function=preprocess_input,  
    rotation_range=20,  
    zoom_range=0.2,  
    horizontal_flip=True  
)
```

Dataset Split: 80% Train | 20% Validation

Optimizer & Compilation

Using categorical crossentropy for multi-class classification labels.

```
model.compile(  
    optimizer=Adam(learning_rate=0.0001),  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

- Loss: Categorical Crossentropy
- LR: 0.0001 (Adaptive)
- Epochs: 20 with Early Stopping

Projected Outcome: Automated detection with High-Accuracy lesion stratification.